arXiv:1811.09030v1 [cs.CV] 22 Nov 2018

Data Augmentation using Random Image Cropping and Patching for Deep CNNs

Ryo Takahashi, Takashi Matsubara, *Member, IEEE*, and Kuniaki Uehara,

***Abstract*—Deep convolutional neural networks (CNNs) have achieved remarkable results in image processing tasks. However, their high expression ability risks overfitting. Consequently, data augmentation techniques have been proposed to prevent overfitting while enriching datasets. Recent CNN architectures with more parameters are rendering traditional data augmentation techniques insufficient. In this study, we propose a new data augmentation technique called *random image cropping and patching* (*RICAP*) which randomly crops four images and patches them to create a new training image. Moreover, RICAP mixes the class labels of the four images, resulting in an advantage similar to label smoothing. We evaluated RICAP with current state-of-the-art CNNs (e.g., the shake-shake regularization model) by comparison with competitive data augmentation techniques such as cutout and mixup. RICAP achieves a new state-of-the-art test error of 2.19% on CIFAR-10. We also confirmed that deep CNNs with RICAP achieve better results on classification tasks using CIFAR-100 and ImageNet and an image-caption retrieval task using Microsoft COCO.**

***Index Terms*—Data Augmentation, Image Classification, Convolutional Neural Network, Image-Caption Retrieval**

I. Introduction

Deep convolutional neural networks (CNNs) [1] have led to significant achievement in the fields of image classification and image processing owing to their numerous parameters and rich expression ability [2], [3]. A recent study demonstrated that the performance of CNNs is logarithmically proportional to the number of training samples [4]. Conversely, without enough training samples, CNNs with numerous parameters have a risk of overfitting because they memorize detailed features of training images that cannot be generalized [2], [5]. Since collecting numerous samples is prohibitively costly, data augmentation methods have been commonly used [6], [7]. Data augmentation increases the variety of images by manipulating them in several ways such as flipping, resizing, and random cropping [8]–[11]. Color jitter changes the brightness, contrast, and saturation, and color translating alternates intensities of RGB channels using principal component analysis (PCA) [12]. Dropout [13] is a common data augmentation technique that injects noise into an image by dropping pixels. Unlike conventional data augmentation techniques, dropout can disturb and mask the features of original images. Many recent studies have proposed new CNN architectures that have many more parameters [14]–[18], and the above traditional data augmentation techniques have become insufficient.

R. Takahashi, T. Matsubara, and K. Uehara are with Graduate School of System Informatics, Kobe University, 1-1 Rokko-dai, Nada, Kobe, Hyogo, 657-8501 Japan. E-mails: takahashi@ai.cs.kobe-u.ac.jp, matsubara@phoenix.kobe-u.ac.jp, and uehara@kobe-u.ac.jp.

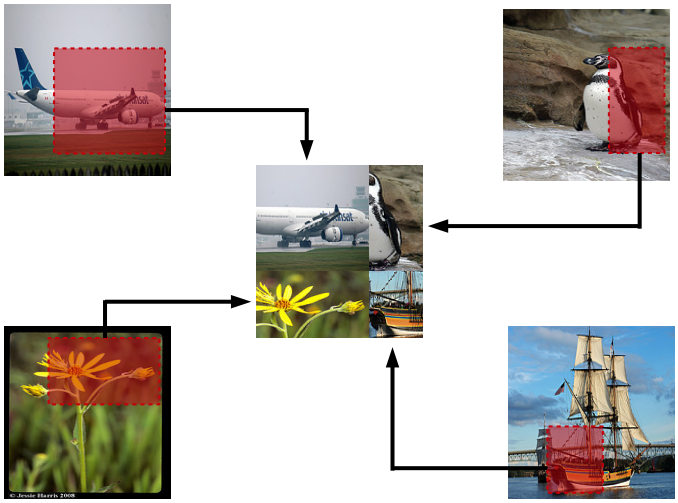


Fig. 1. Conceptual explanation of the proposed *random image cropping and patching* (*RICAP*) data augmentation. Four training images are randomly cropped as denoted by the red shaded areas, and patched to construct a new training image (at center). The size of the final image is identical to that of the original one (e.g., 32×32 for the CIFAR dataset [8]). These images are collected from the training set of the ImageNet dataset [23].

Therefore, nowadays, new data augmentation techniques have attracted increasing attention [19]–[21]. Cutout [19] randomly masks a square region in an image at every training step and thus changes the apparent features. Cutout is an extension of dropout that can achieve better performance. Random erasing [20] also masks a subregion in an image like cutout. Unlike cutout, it randomly determines whether to mask a region as well as the size and aspect ratio of the masked region. Mixup [21] alpha-blends two images to form a new image, regularizing the CNN to favor simple linear behavior in-between training images. In addition to an increase in the variety of images, mixup behaves like class label smoothing as it mixes the class labels of two images with the ratio $\lambda : 1 - \lambda$ [22]. These new data augmentation techniques have been applied to modern deep CNNs and have broken records, demonstrating the importance of data augmentation.

In this study, as a further advancement in data augmentation, we propose a novel method called *random image cropping and patching* (*RICAP*). RICAP crops four training images and patches them to construct a new training image; it selects images and determines the cropping sizes randomly, where the size of the final image is identical to that of the original image. A conceptual explanation is shown in Fig. 1. RICAP also mixes class labels of the four images with ratios proportional

to the areas of the four images like label smoothing in mixup. Compared to mixup, RICAP has three clear distinctions: it mixes images spatially, it uses partial images by cropping, and it does not create features that are absent in the original dataset except for boundary patching.

We introduce the detailed algorithm of RICAP in Section III-A and explain its conceptual contributions in Section III-B. We apply RICAP to existing deep CNNs and evaluate them on the classification tasks using the CIFAR-10, CIFAR-100 [8], and ImageNet [23] datasets in Sections IV-A, IV-B and IV-C. The experimental results demonstrate that RICAP outperforms existing data augmentation techniques, In particular, RICAP achieves a new state-of-the-art performance on CIFAR-10 classification task. Furthermore, we visualize the region where the model focuses attention using class activation mapping [24] in Section IV-D, demonstrating that RICAP makes CNNs focus attention on various objects and features in an image, in other words, RICAP prevents CNNs from overfitting to specific features. In addition, we confirm that RICAP works well fpr an image-caption retrieval task using Microsoft COCO dataset [25] in Section IV-E. We describe the ablation study performed in Section IV-F and make further comparison of RICAP with mixup in Section IV-G.

Limited preliminary results can be found in a recent conference proceeding [26].

II. Related Works

RICAP is a novel data augmentation technique and can be applied to deep CNNs in the same manner as conventional techniques. In addition, RICAP is related to the class label smoothing technique. In this section, we explain related works on data augmentation and label smoothing.

A. Data Augmentation

Data augmentation increases the variety of training samples and prevents overfitting [6], [7]. A deep CNN, AlexNet [12], used random cropping and horizontal flipping for evaluation on the CIFAR dataset [8]. Random cropping prevents a CNN from overfitting to specific features by changing the apparent features in an image. Horizontal flipping doubles the variation in an image with specific orientations, such as a side-view of an airplane. AlexNet also performed principal component analysis (PCA) on a set of RGB values to alter the intensities of the RGB channels for evaluation on the ImageNet dataset [23]. They added multiples of the found principal components to each image. This type of color translation is useful for colorful objects, such as flowers. Facebook AI Research employed another method of color translation called color jitter for the reimplementation of ResNet [11] available at https://github.com/facebook/fb.resnet.torch. Color jitter randomly changes the brightness, contrast, and saturation of an image instead of the RGB channels. These traditional data augmentation techniques play an important role in training deep CNNs. However, the number of parameters is ever-growing and the risk of overfitting is also ever-increasing as many studies propose new network architectures [14]–[18] following ResNet[11]. Therefore, data augmentation techniques have attracted further attention.

Dropout [13] is a data augmentation technique that disturbs and masks the original information of given data by dropping pixels. Pixel dropping functions as injection of noise into an image [27]. It makes the CNN robust to noisy images and contributes to generalization rather than enriching the dataset.

Cutout randomly masks a square region in an image at every training step [19]. It is an extension of dropout, where masking of regions behaves like injected noise and makes CNNs robust to noisy images. In addition, cutout can mask the entire main part of an object in an image, such as the face of a cat. In this case, CNNs need to learn other parts that are usually ignored, such as the tail of the cat. This prevents deep CNNs from overfitting to features of the main part of an object. A similar method, random erasing, has been proposed [20]. It also masks a certain area of an image but has clear differences; it randomly determines whether to mask a region as well as the size and aspect ratio of the masked region.

Mixup alpha-blends two images to construct a new training image [21]. Mixup can train deep CNNs on convex combinations of pairs of training samples and their labels, and enables deep CNNs to favor a simple linear behavior in-between training samples. This behavior makes the prediction confidence transit linearly from one class to another class, thus providing smoother estimation and margin maximization. Alpha-blending not only increases the variety of training images but also works like adversarial perturbation [28]. Thereby, mixup makes deep CNNs robust to adversarial examples and stabilizes the training of generative adversarial networks. In addition, it behaves similar to class label smoothing by mixing class labels with the ratio $\lambda : 1 - \lambda$ [22]. We explain label smoothing in detail below.

AutoAugment [29] is a framework exploring the best hyperparameters of existing data augmentations using reinforcement learning [30]. It achieved significant results on the CIFAR-10 classification and proved the importance of data augmentation for the learning of deep CNN.

B. Label Smoothing

In classification tasks, class labels are often expressed as probabilities of 0 and 1. Deep CNNs commonly employ the softmax function, which never predicts an exact probability of 0 and 1. Thus, deep CNNs continue to learn increasingly larger weight parameters and make an unjustly high confidence. Label smoothing sets the class probabilities to intermediate values, such as 0.9 and 0.8. It prevents the endless pursuit of hard 0 and 1 probabilities for the estimated classes and enables the weight parameters to converge to certain values without discouraging correct classification [22]. Mixup mixes class labels of the blended images with the ratio $\lambda : 1 - \lambda$ and has a similar contribution to label smoothing [21].

III. Proposed Method

A. Random Image Cropping and Patching (RICAP)

In this study, we propose a novel data augmentation technique called *random image cropping and patching* (*RICAP*) for deep convolutional neural networks (CNNs). The conceptual explanation of RICAP is shown in Fig. 1. It consists of

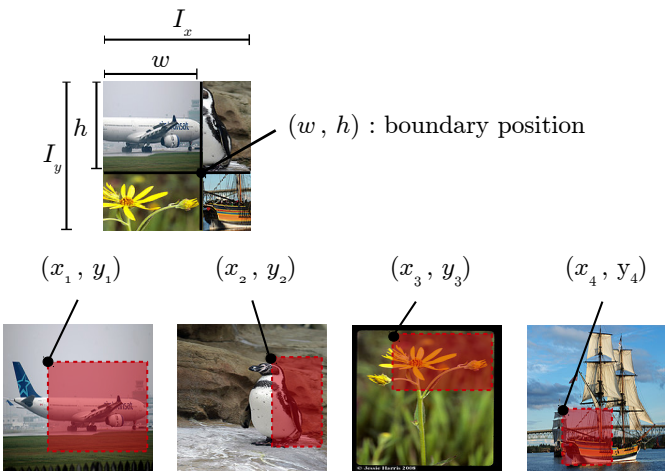


Fig. 2. Detailed explanation of RICAP. I_x and I_y are the width and height of the original image, respectively. Four images are randomly cropped, as denoted by the red shaded areas, and patched according to the boundary position (w, h). The boundary position (w, h) is generated from a beta distribution Beta(β, β), where β is a hyperparameter of RICAP. Based on the boundary position (w, h), the cropped positions (x_k, y_k) are selected such that they do not change the image size.

three data manipulation steps. First, four images are randomly selected from the training set. Second, the images are cropped separately. Third, the cropped images are patched to create a new image. Despite this simple procedure, RICAP increases the variety of images drastically and prevents overfitting of deep CNNs having numerous parameters. The class labels of the four images are mixed with ratios proportional to the image areas. This label mixing works as label smoothing and prevents the endless pursuit of the hard 0 and 1 probabilities in deep CNNs using the softmax function.

A more specific explanation of the implementation is shown in Fig. 2. We randomly select four images $k \in \{1, 2, 3, 4\}$ from the training set and patch them on the upper left, upper right, lower left, and lower right regions. I_x and I_y denote the width and height of the original training image, respectively. (w, h) is the boundary position which gives the size and position of each cropped image. We choose this boundary position (w, h) in every training step from beta distributions as below.

$$w = \mathrm{round}(w' I_x), \quad h = \mathrm{round}(h' I_y),$$
$$w' \sim \mathrm{Beta}(\beta, \beta), \quad h' \sim \mathrm{Beta}(\beta, \beta),$$

where $\beta \in (0, \infty)$ is a hyperparameter and $\mathrm{round}(\cdot)$ is the rounding function. Once we determine the boundary position (w, h), we automatically obtain the cropping sizes (w_k, h_k) of the images k, i.e., $w_1 = w_3 = w$, $w_2 = w_4 = I_x - w$, $h_1 = h_2 = h$, and $h_3 = h_4 = I_y - h$. For cropping the four images k following the sizes (w_k, h_k), we randomly determine the positions (x_k, y_k) of the upper left corners of the cropped areas as

$$x_k \sim \mathcal{U}(0, I_x - w_k),$$
$$y_k \sim \mathcal{U}(0, I_y - h_k).$$

Finally, we define the target label c by mixing one-hot coded class labels c_k of the four patched images with ratios W_i proportional to their areas in the new constructed image;

$$c = \sum_{k \in \{1,2,3,4\}} W_k c_k \ \text{ for } \ W_k = \frac{w_k h_k}{I_x I_y}, \tag{1}$$

where $w_k h_k$ is the area of the cropped image k and $I_x I_y$ is the area of the original image.

The hyperparameter β determines the distribution of boundary position. If β is large, the boundary position (w, h) tends to be close to the center of a patched image and the target class probabilities c often have values close to 0.25. RICAP encounters a risk of excessive label smoothing, discouraging correct classification. If β is small, the boundary position (w, h) tends to be close to the four corners of the patched image and the target class probabilities c often have 0 or 1 probabilities. Especially, with $\beta = 0$, RICAP does not augment images but provides original images. With $\beta = 1.0$, the boundary position (w, h) is distributed uniformly over the patched images. For reproduction, we provide a Python code of RICAP in Algorithm 1 in Appendix.

B. Concept of RICAP

RICAP shares concepts with cutout, mixup, and label smoothing, and potentially overcomes their shortcomings.

Cutout masks a subregion of an image and RICAP crops a subregion of an image. Both change the apparent features of the image at every training step. However, masking in cutout simply reduces the amount of available features in each sample. Conversely, the proposed RICAP patches images, and hence the whole region of a patched image produces features contributing to the training.

Mixup employs an alpha-blend (i.e., blending of pixel intensity), while RICAP patches four cropped images, which can be regarded as a spatial blend. By alpha-blending two images, mixup generates pixel-level features that original images never produce, drastically increasing the variety of features that a CNN has to learn and potentially disturbing the training. Conversely, images patched by the RICAP method always produce pixel-level features that original images also produce except for boundary patching.

When the boundary position (w, h) is close to the four corners, a cropped area becomes small and occasionally depicts no object. RICAP does not check whether an object is in the cropped area. Even if an object is absent in the cropped area, a CNN learns other objects from the other cropped areas and enjoys the benefits of label smoothing.

IV. Experiments and Results

To evaluate the performance of RICAP, we apply it to deep CNNs and evaluate on the CIFAR-10, CIFAR-100, and ImageNet datasets in Sections IV-A, IV-B and IV-C. We visualize the regions in which the CNN focuses attention to understand the operation of RICAP in Section IV-D. In addition to classification, we evaluate RICAP on an image-caption retrieval task in Section IV-E. Also, we perform an ablation study in Section IV-F and a further comparison of RICAP with mixup in Section IV-G.

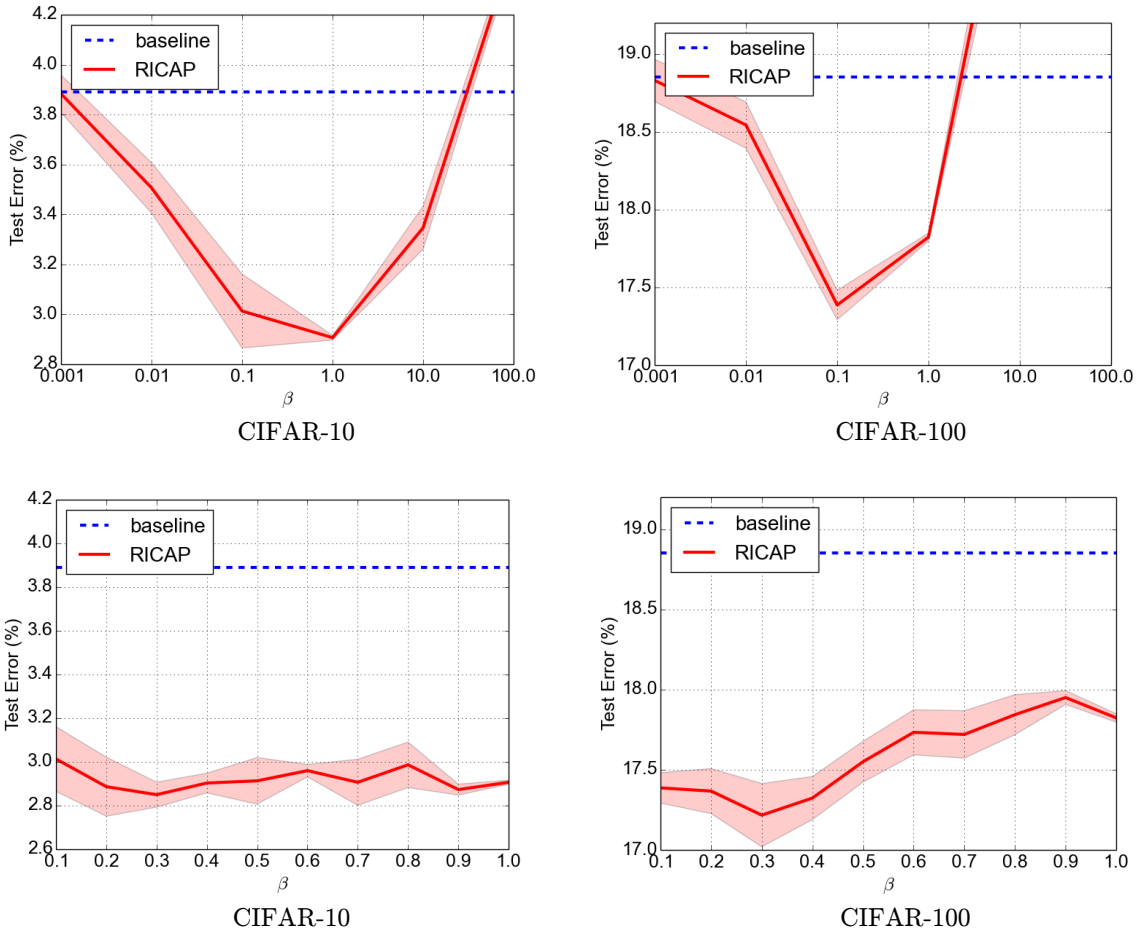


Fig. 3. Exploration of the hyperparameter β of RICAP using the WideResNet 28-10 for a wider range of β on CIFAR-10 (left upper panel) and on CIFAR-100 (right upper panel) and for a more specific range of $[0.1, 1.0]$ on CIFAR-10 (left lower panel) and on CIFAR-100 (right upper panel). We performed three runs, depicting the means and standard deviations by solid lines and shaded areas, respectively. The baseline indicates the results of the WideResNet without RICAP.

A. *Classification of CIFAR-10 and CIFAR-100*

In this section, we show the application of RICAP to an existing deep CNN and evaluate it on the classification tasks of the CIFAR-10 and CIFAR-100 datasets [8]. CIFAR-10 and CIFAR-100 consist of 32×32 RGB images of objects in natural scenes. $50,000$ images are used for training and $10,000$ for evaluation. Each image is manually assigned one of the 10 class labels in CIFAR-10 and one of the 100 in CIFAR-100. The number of images per class is thus reduced in CIFAR-100. Based on previous studies [31]–[33], we normalized each channel of all images to zero mean and unit variance as preprocessing. We also employed 4-pixel padding on each side, 32×32 random cropping, and random flipping in the horizontal direction as conventional data augmentation techniques.

We used a residual network called WideResNet[15]. We used an architecture called *WideResNet 28-10*, which consists of 28 convolution layers with a widen factor of 10 and employs dropout with a drop probability of $p = 0.3$ in the intermediate layers. This architecture achieved the highest accuracy on the CIFAR datasets in the original study [15]. The hyperparameters were set to be the same as those used in the original study. Batch normalization [34] and ReLU activation function [35] were used. The weight parameters were initialized following the He algorithm [36]. The weight parameters were updated using the momentum SGD algorithm with a momentum parameter of 0.9 and weight decay of 10^{-4} over 200 epochs with batches of 128 images. The learning rate was initialized to 0.1, and then, it was reduced to 0.02, 0.004 and 0.0008 at the 60th, 120th and 160th epochs, respectively.

We evaluated RICAP with WideResNet to explore the best value of the hyperparameter β. I_x and I_y were 32 for the CIFAR datasets. Figure 3 shows the results on CIFAR-10 and CIFAR-100. The baselines denote the results of the WideResNet without RICAP. For both CIFAR-10 and CIFAR-100, $\beta = 0.3$ resulted the best test error rates. With an excessively large β, we obtained worse results than the baseline, which suggests the negative influence of excessive label smoothing. With decreasing β, the performance converged to the baseline results. We also summarized the results of RICAP in Table I as well as the results of competitive methods: dropout [13], cutout [19], random erasing [20], and mixup [21]. Com-

TABLE I
TEST ERROR RATES USING WIDERESNET ON THE CIFAR DATASET.

Method	CIFAR-10	CIFAR-100
Baseline	3.89	18.85
+ dropout ($p = 0.2$)	4.65 $\pm$0.08†	21.27 $\pm$0.19†
+ cutout (16×16)	3.08 $\pm$0.16	18.41 $\pm$0.27
+ random erasing	3.08 $\pm$0.05	17.73 $\pm$0.15
+ mixup ($\alpha = 1.0$)	3.02 $\pm$0.04†	17.62 $\pm$0.25†
+ RICAP ($\beta = 0.3$)	**2.85** $\pm$0.06	**17.22** $\pm$0.20

† indicates the results of our experiments.

TABLE II
SINGLE CROP TEST ERROR RATES ON IMAGENET USING THE WIDERESNET-50-2-BOTTLENECK.

Method	Epochs	top-1 Error(%)	top-5 Error(%)
Baseline	100	21.90	6.03
+ cutout (56×56)	100	22.45†	6.22†
+ mixup ($\alpha = 0.2$)	100	21.83†	5.81†
+ RICAP ($\beta = 0.3$)	100	**21.08**	**5.66**
Baseline	200	21.84†	6.03†
+ cutout (56×56)	200	21.51†	5.89†
+ mixup ($\alpha = 0.2$)	200	20.39†	**5.22**†
+ RICAP ($\beta = 0.3$)	200	**20.33**	5.26

† indicates the results of our experiments.

petitive results denoted by † symbols were obtained from our experiments and the other results were cited from the original studies. In our experiments, each value following the $\pm$ symbol was the standard deviation over three runs. Recall that WideResNet usually employs dropout in intermediate layers. As the dropout data augmentation, we added dropout to the input layer for comparison. The drop probability was set to $p = 0.2$ according to the original study [13]. For other competitive methods, we set the hyperparameters to values with which the CNNs achieved the best results in the original studies: cutout size 16×16 (CIFAR-10) and 8×8 (CIFAR-100) for cutout and $\alpha = 1.0$ for mixup. RICAP clearly outperformed the competitive methods.

B. Classification of ImageNet

In this section, we evaluate RICAP on the classification task of the ImageNet dataset [23]. ImageNet consists of 1.28 million training images and 50,000 validation images. Each image is given one of 1,000 class labels. We normalized each channel of all images to zero mean and unit variance as preprocessing. We also employed random resizing, random 224×224 cropping, color jitter, lighting, and random flipping in the horizontal direction following previous studies [15], [21].

To evaluate RICAP, we applied it to the *WideResNet 50-2-bottleneck* architecture, consisting of 50 convolution layers using bottleneck residual blocks with a widen factor of 2 and dropout with a drop probability of $p = 0.3$ in intermediate layers [15]. This architecture achieved the highest accuracy on ImageNet in the original study [15]. The hyperparameters and other conditions were the same as those used in the baseline study. WideResNet 50-2-bottleneck was trained using the momentum SGD algorithm with a momentum parameter of 0.9 and weight decay of 10^{-4} over 100 or 200 epochs with batches of 256 images. The learning rate was initialized to 0.1, and then, it was reduced to 0.01, 0.001 and 0.0001 at the 30th, 60th, and 90th-epoch, respectively, in the case of 100 epoch training. The learning rate was reduced at the 65th, 130th, and 190th-epoch, respectively, in the case of 200 epoch training. For our RICAP, we used the hyperparameter $\beta = 0.3$ according to the results of Section. IV-A.

Table II summarizes the results of RICAP with the WideResNet 50-2-bottleneck as well as the results of competitive methods: cutout [19] and mixup [21]. Competitive results denoted by † symbols were obtained from our experiments and the other results are cited from the original studies. We used $\alpha = 0.2$ for mixup according to the original study. Cutout did not attempt to apply cutout to the ImageNet dataset. It used a cutout size of 8×8 for the CIFAR-10, in which an image has a size of 32×32. Since a preprocessed image in the ImageNet dataset has a size of 224×224, we multiplied the cutout size by 7 $(224/32)$ to apply cutout to the ImageNet dataset.

RICAP clearly outperformed the baseline and competitive methods in the case of 100 epoch training, and was superior or competitive to the others in the case of 200 epoch training. Compared to RICAP, cutout and mixup require a longer training to get results better than the baseline. This is because, as mentioned in Section III-B, cutout reduces the amount of available features in each and mixup generates pixel-level features that original images never produce.

Mixup requires careful adjustment of the hyperparameter; the best hyperparameter value is $\alpha = 1.0$ for the CIFAR datasets and $\alpha = 0.2$ for the ImageNet dataset. An inappropriate hyperparameter reduces performance significantly [21]. On the other hand, RICAP with the hyperparameter $\beta = 0.3$ achieved significant results in both the CIFAR and ImageNet datasets. Futhermore, the bottom panels in Fig. 3 show the robustness of RICAP to the hyperparameter value.

C. Classification by Other Architectures

We also evaluated RICAP with DenseNet [16], the pyramidal ResNet [17], and the shake-shake regularization model [37] on the CIFAR-10 dataset [8]. For the DenseNet, we used the architecture *DenseNetBC 190-40*; as the name implies, it consists of 190 convolution layers using bottleneck residual blocks with a growing rate of 40. For the pyramidal ResNet, we used the architecture *Pyramidal ResNet 272-200*, which consists of 272 convolution layers using bottleneck residual blocks with a widening factor of 200. For the shake-shake regularization model, we used the architecture *ShakeShake 26 2×96d*; this is a ResNet with 26 convolution layers and 2×96d channels with shake-shake image regularization. These architectures achieved the best results in the original studies.

TABLE III
TEST ERROR RATES ON CIFAR-10.

Method	DenseNet-BC 190-40	Pyramidal ResNet 272-200	Shake-Shake 26 2x96d
Baseline	3.46	3.31 $\pm$0.08	2.86
+ dropout ($p = 0.2$)	4.56 †	4.06 †	3.79 †
+ cutout (8×8)	2.73 $\pm$0.06†	2.84 $\pm$0.05†	2.56 $\pm$0.07
+ mixup ($\alpha = 1.0$)	2.73 $\pm$0.08†	2.57 $\pm$0.09†	2.32 $\pm$0.11†
+ RICAP ($\beta = 0.3$)	**2.69** $\pm$0.12	**2.51** $\pm$0.02	**2.19** $\pm$0.08

† indicates the results of our experiments.

We applied data normalization and data augmentation in the same way as Section. IV-A. The hyperparameters were the same as those in the original studies [16], [17], [37].

We summarized the results in Table III. We used the hyperparameter $\beta = 0.3$ according to the results of Section. IV-A. RICAP outperformed the competitive methods. In particular, the shake-shake regularization model with RICAP achieved a test error rate of 2.19%; this is a new record on the CIFAR-10 classification among the studies under the same conditions [16], [17], [19]–[21], [37][1]. These results also indicate that RICAP is applicable to various CNN architectures and the appropriate hyperparameter does not depend on the CNN architectures.

D. Visualization of Feature Learning by RICAP

One of the most serious overfitting of a CNN arises when classifying images according a limited set of features and ignoring others. For example, if a CNN classifies cat images according to features of the cats' face, it fails to classify an image that depicts a cats' back. Since RICAP collects and crops four images randomly, each image provides a different cropped region in every training step. This is expected to support the CNN in using a wider variety of features from the same image and to prevent the CNN from overfitting to features of a specific region.

To verify this hypothesis, we visualized the regions in which a CNN focuses much attention using the *Class Activation Mapping (CAM)* [24]. The CAM expects a CNN to have a global average pooling layer to obtain the spatial average of the feature map before the final output layer. The CAM calculates the regional importance by projecting back the output (typically, the correct label) to the feature map.

Figure 4 shows the CAMs of the WideResNet 50-2-bottleneck with and without RICAP. This model was trained in the previous ImageNet experiments in Section IV-B. The top row shows the input images. The middle row denoted as the baseline shows the CAMs of WideResNet without RICAP. WideResNet focuses attention on limited regions of objects in the first to sixth columns: the shell of a turtle and the faces of animals. WideResNet focuses attention on objects in the foreground and ignores objects in the background in the seventh and tenth columns. The bottom row shows the CAMs of WideResNet with RICAP. WideResNet focuses attention on the whole bodies of animals in the first to sixth columns and objects in the foreground and background in the seventh to tenth columns. These results demonstrate that RICAP prevents the CNN from overfitting to specific features.

In addition, we visualized the CAMs of the WideResNet using the images that RICAP cropped and patched in Fig. 5. The leftmost column shows the input images. The second to fifth columns show the CAMs obtained by projecting back the labels corresponding to the upper left, upper right, lower left, and lower right image patches, respectively. The CAMs demonstrated that WideResNet focuses attention on the object corresponding to each given label correctly, even though the depicted objects were extremely cropped. Moreover, we confirmed that WideResNet automatically learns to ignore the boundary patching caused by RICAP and potentially becomes robust to occlusion and cutting off.

E. Evaluation on Image-Caption Retrieval

Image-Caption Retrieval: In this section, we evaluate RICAP with a task other than classification, i.e., image-caption retrieval. For image-caption retrieval, the main goal is to retrieve the most relevant image for a given caption and to retrieve the most relevant caption for a given image. A dataset contains pairs of images i_n and captions c_n. An image i_n is considered the most relevant to the paired caption c_n and vice versa. A relevant pair (i_n, c_n) is called positive, and an irrelevant pair (i_n, c_m) $(m \neq n)$ is called negative. The performance is often evaluated using recall at K (denoted as $R@K$) and $Med\ r$.

A common approach for image-caption retrieval is called *visual-semantic embeddings (VSE)* [38]. Typically, a CNN encodes an image into a vector representation and a recurrent neural network (RNN) encodes a caption to another vector representation. The neural networks are jointly trained to build a similarity function that gives higher scores to positive pairs than negative pairs. *VSE++* [39] employed a ResNet152 [11] as the image encoder and a GRU [38] as the caption encoder and achieved remarkable results. We used VSE++ as a baseline of our experiments.

First, we introduce the case without RICAP. An image i_n is fed to the image encoder $ResNet(\cdot)$ and encoded to a

[1]AutoAugment [29] achieved a further improved result by employing additional data augmentation techniques such as shearing and adjusting their parameters.

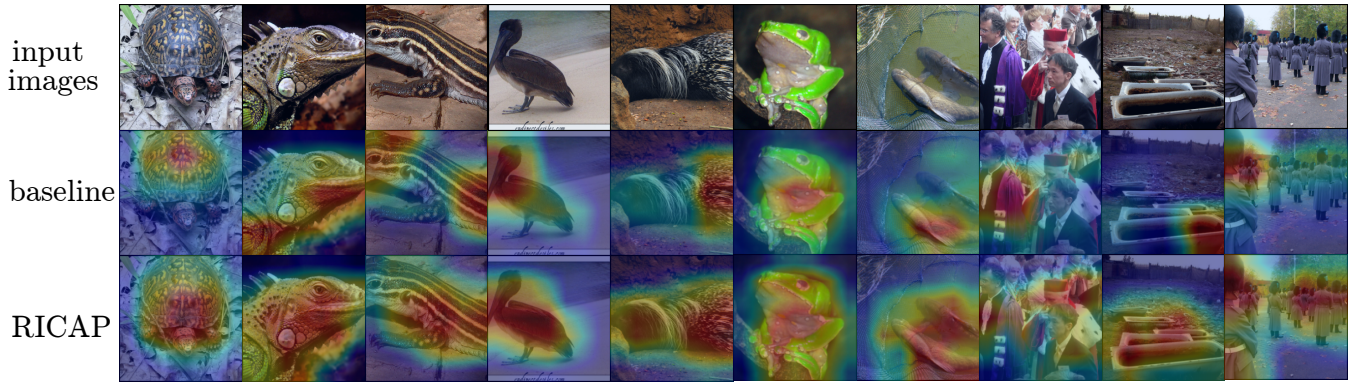


Fig. 4. Class Activation Mapping (CAM) [24] of WideResNet 28-10. The top row shows the input images. The middle row shows the CAM of WideResNet 28-10 without RICAP denoted as *baseline*. The bottom row shows the CAM of WideResNet 28-10 with RICAP.

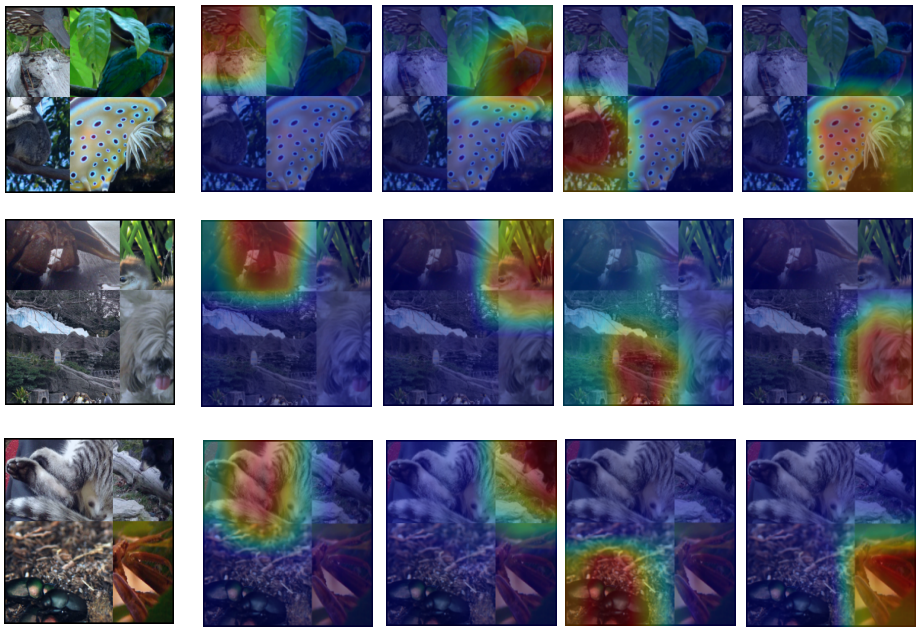


Fig. 5. Class Activation Mapping (CAM) [24] of WideResNet 28-10 using the images that RICAP cropped and patched. The leftmost column shows the input images. The second to fifth columns show the CAMs by projecting back the labels corresponding to the upper left, upper right, lower left, and lower right image patches, respectively.

TABLE IV
RESULTS OF IMAGE-CAPTION RETRIEVAL USING MS-COCO.

Model	Caption Retrieval				Image Retrieval			
	R@1	R@5	R@10	Med r	R@1	R@5	R@10	Med r
Baseline	64.6	90.0	95.7	**1.0**	52.0	84.3	92.0	**1.0**
+ RICAP ($\beta = 0.3$)	**65.8**	**90.2**	**96.2**	**1.0**	**52.3**	**84.4**	**92.4**	**1.0**

representation v_{i_n} as

$$v_{i_n} = ResNet(i_n).$$

A caption c_n is fed to the caption encoder $GRU(\cdot)$ and encoded to a representation v_{c_n} as

$$v_{c_n} = GRU(c_n).$$

VSE++ $S(\cdot,\cdot)$ defines the similarity S_n between the pair (i_n, c_n) as

$$\begin{aligned} S_n &= S(v_{i_n}, v_{c_n}), \\ &= S(ResNet(i_n), GRU(c_n)). \end{aligned}$$

Refer to the original study [39] for a more detailed implementation.

Application RICAP to VSE++: With RICAP, we propose a new training procedure for the image encoder. We randomly selected four images i_m, i_n, i_o, and i_p and created a new image i_{ricap} as the cropping and patching procedure in Section. III.

$$i_{ricap} = RICAP_{image}(i_m, i_n, i_o, i_p).$$

The function $RICAP_{image}(\cdot,\cdot,\cdot,\cdot)$ denotes the procedure of RICAP proposed in Section III-A. The image encoder $ResNet(\cdot)$ encodes the patched image i_{ricap} to a representation $v_{i_{ricap}}$ as

$$v_{i_{ricap}} = ResNet(i_{ricap}).$$

As a paired caption representation, we obtained the average of the relevant caption representations like the label smoothing. The specific procedure was as follows. We fed the paired captions c_m, c_n, c_o, and c_p individually to the caption encoder and encoded them into the representations v_{c_m}, v_{c_n}, v_{c_o}, and v_{c_p}, respectively. Next, we averaged the caption representations v_{c_m}, v_{c_n}, v_{c_o}, and v_{c_p} with ratios proportional to the areas of the cropped images and obtained the mixed vector $v_{c_{ricap}}$ as

$$\begin{aligned} v_{c_{ricap}} &= RICAP_{caption}(v_{c_m}, v_{c_n}, v_{c_o}, v_{c_p}) \\ &:= \textstyle\sum_{k=\{m,n,o,p\}} W_k v_{c_k}, \\ &= \textstyle\sum_{k=\{m,n,o,p\}} W_k GRU(c_k), \end{aligned}$$

where W_k is the area ratio of the image k as in Eq. (1). Here, we used the vector representation $v_{c_{ricap}}$ as the one positively paired with the vector representation $v_{i_{ricap}}$ and obtained the similarity S_{ricap} between this pair. In short, we used the following similarity to train the image encoder;

$$\begin{aligned} &S_{ricap} \\ &= S(v_{i_{ricap}}, v_{c_{ricap}}), \\ &= S(ResNet(RICAP_{image}(i_m, i_n, i_o, i_p)), \\ &\quad RICAP_{caption}(GRU(c_m), GRU(c_n), GRU(c_o), GRU(c_p))). \end{aligned}$$

We treated the remaining vector representations v_{c_k} for $k \notin \{m, n, o, p\}$ as negative pairs. Note that we used the ordinary similarity S_n to train the caption encoder.

Experiments and Results: We used the same experimental settings as in the original study of VSE++ [39]. We used the Microsoft COCO dataset [25]; $113,287$ images for training model and $1,000$ images for validation. We summarized the score averaged over 5 folds of $1,000$ test images.

The ResNet was pre-trained using the ImageNet dataset, and the final layer was replaced with a new fully-connected layer. For the first 30 epochs, the layers in the ResNet except for the final layer were fixed. The GRU and the final layer of the ResNet were updated using the Adam optimizer [40] using a mini-batch size of 128 with the hyperparameter $\alpha = 0.0002$ for the first 15 epochs and then $\alpha = 0.00002$ for the other 15 epochs. Next, the whole model was fine-tuned for the additional 15 epochs with $\alpha = 0.00002$.

Table IV summarizes the results; RICAP improved the performance of VSE++. This result demonstrated that RICAP is directly applicable to image processing tasks other than classification.

F. Ablation Study

RICAP is composed of two manipulations: image mixing and label mixing. For image mixing, RICAP randomly selects, crops, and patches four images to construct a new training image. For label mixing, RICAP mixes class labels with ratios proportional to the areas of four images. In this section, we evaluated the contributions of these two manipulations using *WideResNet 28-10* and the CIFAR-10 and CIFAR-100 datasets [8] as in Section. IV-A. Data normalization, data augmentation, and the hyperparameters were also the same as those used in Section. IV-A. We chose the hyperparameter β of RICAP from $\{0.1, 0.3, 1.0\}$. The results are summarized in Table. V.

First, we performed image mixing without label mixing. We used the class label of the patched image that had the largest area as the target class label, i.e., $c = c_k$ for $k = \arg\max_{k' \in \{1,2,3,4\}} W_{k'}$ instead of Eq. (1). Using only image mixing, WideResNet achieved much better results than the baseline but was not competitive in the case with label mixing.

Second, we performed label mixing without image mixing. In this case, we used the boundary position to calculate only the ratio of label mixing and we used the original image with the largest probability as the training sample. WideResNet achieved much worse results, demonstrating the harmful influence of extreme label smoothing.

We conclude that both image and label mixing jointly play an important role in RICAP.

G. Comparison with Mixup of four Images

RICAP patches four images spatially and mixes class labels using the areas of the patched images. One can find a similarity between RICAP and mixup; mixup alpha-blends two images and mixes their class labels using the alpha value. The main difference between these two methods is that between spatially patching and alpha-blending. Another difference is the number of mixed images: four for RICAP and two for mixup.

Here, as a simple extension of mixup, we evaluated mixup that mixes four images and call it 4-mixup. In this experiment,

TABLE V
TEST ERROR RATES USING WIDERESNET IN THE ABLATION STUDY.

Method	CIFAR-10	CIFAR-100
Baseline	3.89	18.85
+ mixup ($\alpha = 1.0$)	3.02 $\pm$0.04†	17.62 $\pm$0.25†
+ RICAP (image mixing only, $\beta = 0.1$)	3.34 $\pm$0.09	17.87 $\pm$0.22
+ RICAP (image mixing only, $\beta = 0.3$)	3.33 $\pm$0.10	17.95 $\pm$0.13
+ RICAP (image mixing only, $\beta = 1.0$)	3.70 $\pm$0.07	18.90 $\pm$0.24
+ RICAP (label mixing only, $\beta = 0.1$)	69.28	-
+ RICAP (label mixing only, $\beta = 0.3$)	62.84	-
+ RICAP (label mixing only, $\beta = 1.0$)	68.91	-
+ 4 mixup ($\beta = 0.1$)	3.29 $\pm$0.07†	17.62 $\pm$0.21†
+ 4 mixup ($\beta = 0.3$)	3.11 $\pm$0.05†	18.04 $\pm$0.16†
+ 4 mixup ($\beta = 1.0$)	3.71 $\pm$0.17†	19.57 $\pm$0.15†
+ RICAP ($\beta = 0.1$)	3.01 $\pm$0.15	17.39 $\pm$0.09
+ RICAP ($\beta = 0.3$)	**2.85** $\pm$0.06	**17.22** $\pm$0.20
+ RICAP ($\beta = 1.0$)	2.91 $\pm$0.01	17.82 $\pm$0.03

† indicates the results of our experiments.

we used the *WideResNet 28-10* and the CIFAR-10 and CIFAR-100 datasets [8] as in Section. IV-A. Data normalization, data augmentation, and hyperparameters were also the same as those used in Section. IV-A. The alpha values were chosen in the same way as RICAP with the hyperparameter β.

We summarized the results in Table V. While 4-mixup had better results than the baseline, it had worse results than both the original mixup and RICAP. Increasing the number of images cannot improve the performance of mixup. This suggests that RICAP owes its high performance not to the number of images or to the ability to utilize four images.

V. CONCLUSION

In this study, we proposed a novel data augmentation method called *random image cropping and patching* (*RICAP*) to improve the accuracy of the image classification. RICAP selects four training images randomly, crops them randomly, and patches them to construct a new training image. Experimental results demonstrated that RICAP improves the classification accuracy of various network architectures for various datasets by increasing the variety of training images and preventing overfitting. The visualization results demonstrated that RICAP prevents deep CNNs from overfitting to the most apparent features. The results on the image-caption retrieval task demonstrated that RICAP is applicable to image processing tasks other than classification.

ACKNOWLEDGMENT

This study was partially supported by the MIC/SCOPE #172107101.

APPENDIX

For reproduction, we provide a Python code of RICAP in Algorithm 1. The code uses numpy and PyTorch (torch in code) modules and follows a naming convention used in official PyTorch examples.

REFERENCES

[1] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation Applied to Handwritten Zip Code Recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.

[2] M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks," in *Proc. of European Conference on Computer Vision (ECCV2014)*, 2014, pp. 818–833.

[3] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun, "OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks," in *Proc. of International Conference on Learning Representations (ICLR2014)*, 2014, pp. 1–16.

[4] C. Sun, A. Shrivastava, S. Singh, and A. Gupta, "Revisiting Unreasonable Effectiveness of Data in Deep Learning Era," *arXiv*, pp. 1–13, 2017.

[5] L. M. Zintgraf, T. S. Cohen, T. Adel, and M. Welling, "Visualizing Deep Neural Network Decisions: Prediction Difference Analysis," *Proc. of International Conference on Learning Representations (ICLR2017)*, pp. 1–12, 2017.

[6] D. C. Ciresan, U. Meier, J. Masci, L. M. Gambardella, and U. JSchmidhuber, "Flexible, High Performance Convolutional Neural Networks for Image Classification," in *Proc. of the International Joint Conference on Artificial Intelligence (IJCAI 2011)*, 2011, pp. 1237–1242.

[7] D. Cirean, U. Meier, and J. Schmidhuber, "Multi-column Deep Neural Networks for Image Classification," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2012)*, 2012, pp. 3642–3649.

[8] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," *Technical report, University of Toronto*, pp. 1–60, 2009.

[9] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2015)*, vol. 07-12-June, 2015, pp. 1–9.

[10] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *Proc. of International Conference on Learning Representations (ICLR2015)*, 2015, pp. 1–14.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2016)*, 2016, pp. 770–778.

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NIPS2012)*, 2012, pp. 1097–1105.

[13] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," *arXiv*, pp. 1–18, 2012.

[14] K. He and X. Zhang, "Identity mappings in deep residual networks," *Lecture Notes in Computer Science*, vol. 9908, no. 1, pp. 630–645, 2016.

[15] S. Zagoruyko and N. Komodakis, "Wide Residual Networks," *Proc. of the British Machine Vision Conference (BMVC2016)*, pp. 87.1–87.12, 2016.

[16] G. Huang, Z. Liu, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2017)*, 2017, pp. 2261–2269.

[17] D. Han, J. Kim, and J. Kim, "Deep Pyramidal Residual Networks," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2017)*, 2017, pp. 6307–6315.

[18] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2017)*, 2017, pp. 5987–5995.

[19] T. DeVries and G. W. Taylor, "Improved Regularization of Convolutional Neural Networks with Cutout," *arXiv*, pp. 1–8, 2017.

[20] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random Erasing Data Augmentation," *arXiv*, pp. 1–10, 2017.

Algorithm 1. Python code for RICAP

```
beta = 0.3 # hyperparameter
for (images, targets) in loader:

    # get the image size
    I_x, I_y = images.size()[2:]

    # draw a boundary position (w, h)
    w = int(numpy.round(I_x * numpy.random.beta(beta, beta)))
    h = int(numpy.round(I_y * numpy.random.beta(beta, beta)))
    w_ = [w, I_x - w, w, I_x - w]
    h_ = [h, h, I_y - h, I_y - h]

    # select and crop four images
    cropped_images = {}
    c_ = {}
    W_ = {}
    for k in range(4):
        index = torch.randperm(images.size(0))
        x_k = numpy.random.randint(0, I_x - w_[k] + 1)
        y_k = numpy.random.randint(0, I_y - h_[k] + 1)
        cropped_images[k] = images[index][:, :, x_k:x_k + w_[k], y_k:y_k + h_[k]]
        c_[k] = targets[index]
        W_[k] = w_[k] * h_[k] / (I_x * I_y)

    # patch cropped images
    patched_images = torch.cat(
        (torch.cat((cropped_images[0], cropped_images[1]), 2)
         torch.cat((cropped_images[2], cropped_images[3]), 2)),
        3)

    # get output
    outputs = model(patched_images)

    # calculate loss
    loss = sum([W_[k] * F.cross_entropy(outputs, c_[k]) for k in range(4)])

    # optimize
    ...
```


[21] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," in *Proc. of International Conference on Learning Representations (ICLR2018)*, 2018, pp. 1–13.

[22] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. B. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2016)*, 2016, pp. 2818–2826.

[23] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, and C. V. Jan, "ImageNet Large Scale Visual Recognition Challenge," *arXiv*, pp. 1–43, 2014.

[24] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning Deep Features for Discriminative Localization," in *Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR2016)*, 2016, pp. 2921–2929.

[25] T. Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. of European Conference on Computer Vision (ECCV2014)*, 2014, pp. 740–755.

[26] R. Takahashi, T. Matsubara, and K. Uehara, "RICAP : Random Image Cropping and Patching Data Augmentation for Deep CNNs," in *Proc. of Asian Conference on Machine Learning (ACML2018)*, 2018 (accepted).

[27] J. Sietsma and R. J. Dow, "Creating artificial neural networks that generalize," *Neural Networks*, vol. 4, no. 1, pp. 67–79, 1991.

[28] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *Proc. of International Conference on Learning Representations (ICLR2015)*, 2015, pp. 1–11.

[29] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning Augmentation Policies from Data," *arXiv*, pp. 1–14, 2018.

[30] B. Zoph and Q. V. Le, "Neural Architecture Search with Reinforcement Learning," in *Proc. of International Conference on Learning Representations (ICLR2017)*, 2017, pp. 1–16.

[31] C. Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu, "Deeply-supervised nets," in *Proc. of the 18th International Conference on Artificial Intelligence and Statistics (AISTATS2015)*, vol. 2, 2015, pp. 562–570.

[32] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "Fitnets: Hints for Thin Deep Nets," in *Proc. of International Conference on Learning Representations (ICLR2015)*, 2015, pp. 1–13.

[33] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, "Striving for Simplicity: The All Convolutional Net," in *Proc. of International Conference on Learning Representations (ICLR2015)*, 2015, pp. 1–14.

[34] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. of the 32th International Conference on Machine Learning (ICML2015)*, 2015, pp. 448–456.

[35] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," in *Proc. of the 27th International Conference on Machine Learning (ICML2010)*, no. 3, 2010, pp. 807–814.

[36] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proc. of the IEEE International Conference on Computer Vision (ICCV2016)*, vol. 11-18-Dece, 2016, pp. 1026–1034.

[37] X. Gastaldi, "Shake-Shake regularization," in *ICLR Workshop*, 2017, pp. 1–10.

[38] R. Kiros, R. Salakhutdinov, and R. S. Zemel, "Unifying Visual-Semantic Embeddings with Multimodal Neural Language Models," *arXiv*, pp. 1–13, 2014.

[39] F. Faghri, D. J. Fleet, J. R. Kiros, and S. Fidler, "VSE++: Improving Visual-Semantic Embeddings with Hard Negatives," in *Proc. of the British Machine Vision Conference (BMVC2018)*, 2018, pp. 1–13.
[40] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv*, pp. 1–15, 2014.